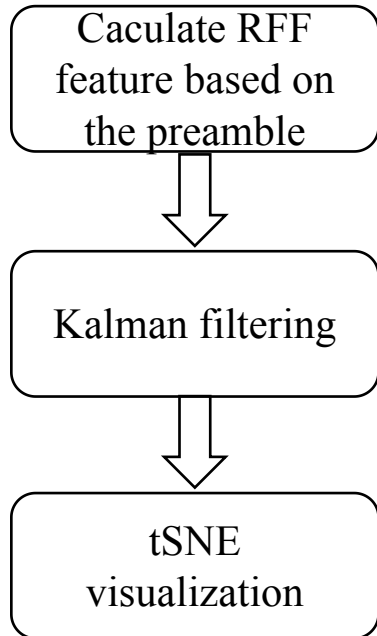


Construct RF features: Caculate RFF feature based on the preamble

- Repository name: RFF_Data_Calculate

<https://www.kaggle.com/code/zeweiguo/rff-data-calculate>



ZEWEI GUO · 6MO AGO · 353 VIEWS

RFF_Data_Calculate

Notebook Input Output Logs Comments (0)

Cite XXXXXXXXXXXXX

Caculate RFF Feature Based on the Preamble

This section extracts ten representative radio-frequency fingerprinting (RFF) features from IEEE 802.11 preamble data. The extracted features include the raw preamble samples, long-preamble I/Q samples, multiple CFO estimations (total, STS-based, and LTS-based), phase and magnitude error metrics, I/Q gain imbalance, and fractal-dimension-based descriptors.

In [1]:

```
import os
import math
import ast
import re
from pathlib import Path
import numpy as np
import pandas as pd

# ----- Utility Functions -----

def distance(a: float, b: float, c: float, d: float) -> float:
    """Euclidean distance between (a,b) and (c,d)."""
    return math.hypot(a - c, d - b)

def power(x: np.ndarray) -> float:
    """Sum of squares."""
    x = np.asarray(x, dtype=float)
```

Version 14 of 14

Runtime: 2h 36m 5s

Input

DATASETS

- RFFI-feature_IQ-wifi-802.11g-2.4G-12
- RFFI-IQ_only-wifi-802.11g-2.4G-123-
- RFFI-kf_feature_IQ-wifi-802.11g-2.4G

Language: Python

Table of Contents

- Caculate RFF Feature Based on the ...
- Kalman Filtering
- tSNE
- Random Forest

Pre-loaded
datasets

Construct RF features: Caculate RFF feature based on the preamble

Caculate RFF Feature Based on the Preamble

This section extracts ten representative radio-frequency fingerprinting (RFF) features from IEEE 802.11 preamble data. The extracted features include the raw preamble samples, long-preamble I/Q samples, multiple CFO estimations (total, STS-based, and LTS-based), phase and magnitude error metrics, I/Q gain imbalance, and fractal-dimension-based descriptors.

Click to
run.

+ Code + Markdown

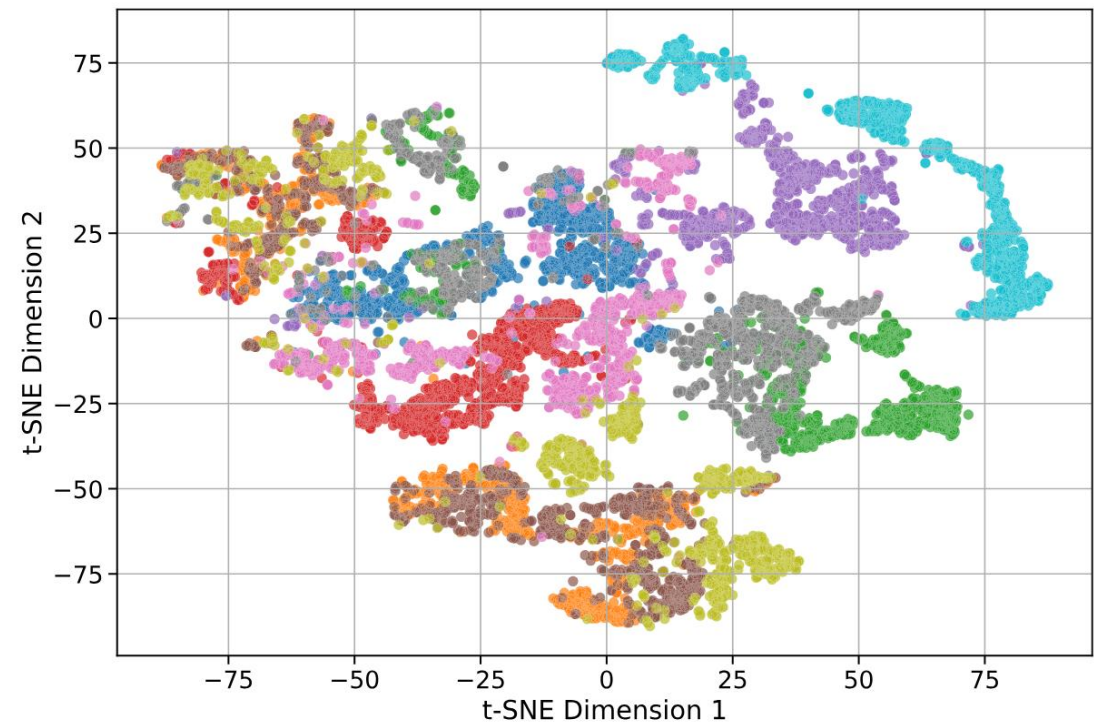
```
import os
import math
import ast
import re
from pathlib import Path
import numpy as np
import pandas as pd

# ----- Utility Functions -----

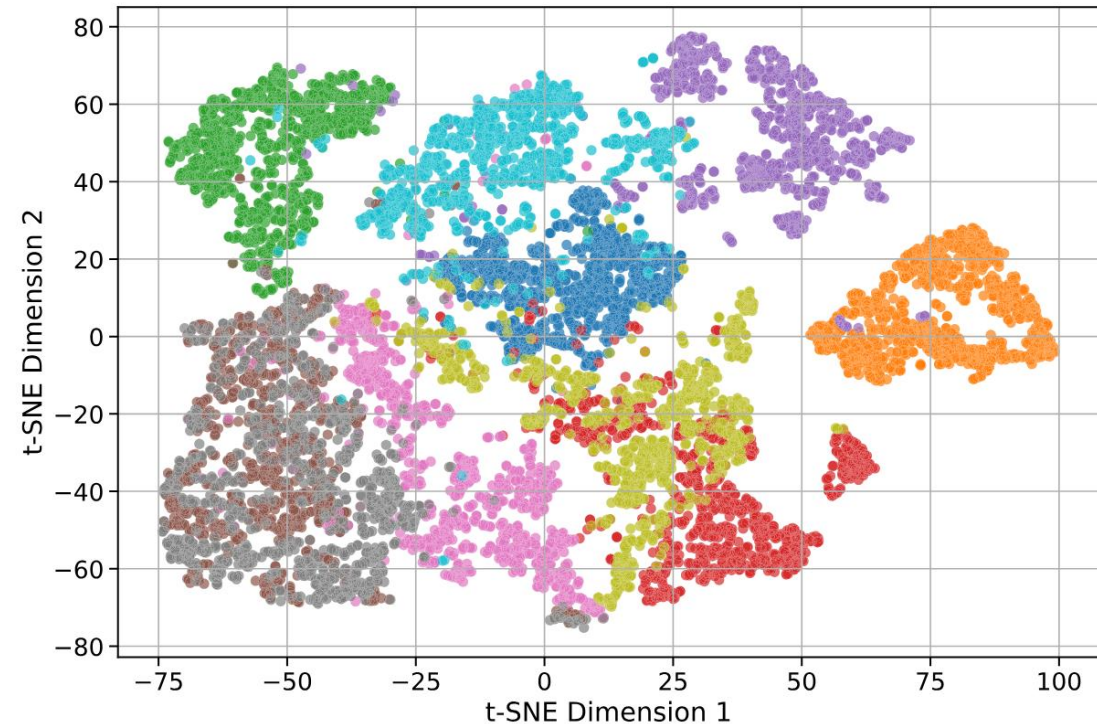
def distance(a: float, b: float, c: float, d: float) -> float:
    """Euclidean distance between (a,b) and (c,d)."""
    return math.hypot(a - c, d - b)

def power(x: np.ndarray) -> float:
    """Sum of squares."""
    x = np.asarray(x, dtype=float)
    return float(np.sum(x * x))
```

Feature visualization



Before Kalman Filtering



After Kalman Filtering

Construct RF features: Kalman filtering

Kalman filtering recursively combines state prediction and noisy observations to obtain a smoother signal estimate.

Kalman Filtering

(You can skip this step and use the pre-processed dataset named "rffi-kf-feature-iq-wifi-802-11g-2-4g-123-m5stack," which allows you to directly perform t-SNE visualization and Random Forest classification.) In this section, the RF features generated in the previous step are refined using Kalman Filtering to improve data smoothness and suppress measurement noise.

Click to
run.



```
import os
import pandas as pd
import numpy as np
try:
    from pykalman import KalmanFilter
except ImportError:
    !pip install pykalman
    from pykalman import KalmanFilter

# Define the Kalman filter function
def apply_kalman_filter(df, columns):
    filtered_data = df.copy()
    for col in columns:
        if col in df.columns:
            try:
```